

## ДОМАШНЕЕ ЗАДАНИЕ 2 · ВТОРАЯ НЕДЕЛЯ

### База знаний и память для вашего агента

Выдано на семинаре 20.09. Дедлайн воскресенье 27.09, 23:59. Сдаётся ссылкой на репозиторий в чат курса, можно продолжать репозиторий первой домашки. Обязательная часть оценивается из десяти баллов, опциональная идёт в зачёт финального проекта.

#### Зачем это задание

В первой домашке агент ходил за каждым фактом в живой поиск: несколько шагов, длинные страницы в контексте, половина вопросов без ответа. Теперь знания лежат рядом, в векторной базе. Проверяем тезис курса на новом месте: сколько стоит верный ответ, когда дешёвая модель получает пять нужных строк вместо всей страницы, и что происходит с ценой диалога, когда агент помнит пользователя, а не таскает за собой всю историю.

## Часть 1, обязательная: RAG и память

### Шаг 1. Корпус и вопросы

Корпус это тексты, по которым ваш агент должен уметь отвечать. Два варианта на выбор:

- 1. Источники вашей первой домашки.** Полные тексты страниц, из которых вы брали вопросы. Как скачать, видно в `fetch_corpus.py` из репозитория семинара. Вопросы с цитатами у вас уже есть.
- 2. Свои документы,** лучше всего те, что нужны вашему проекту: документация, регламенты, статьи, конспекты, переписка из открытого источника. PDF приветствуется. К ним нужно собрать не меньше 30 вопросов с коротким ответом и цитатой.

Требования одинаковые: после нарезки получается не меньше 300 кусков; у каждого вопроса есть поле `evidence` с цитатой из корпуса, по которой определяется верный кусок; у каждого куска есть метаданные, по которым человек найдёт источник: документ, раздел или страница, ссылка. К основным вопросам добавьте **десять вопросов без ответа**: по теме корпуса, но о том, чего в нём нет.

### Шаг 2. Индекс в Milvus и качество поиска

- Две нарезки: наивная по символам и осмысленная, по структуре ваших документов: строки, абзацы, разделы, страницы. Осмысленную опишите одной фразой в отчёте: почему граница куска проходит именно там.
- Эмбединги с кэшем на диске: повторный запуск ноутбука не должен платить за те же тексты второй раз.
- Коллекция в Milvus с метаданными и хотя бы один осмысленный фильтр в демонстрации: по документу, разделу или дате. Milvus поднимается тем же `docker-compose.yml`, что на семинаре: положите его в репозиторий. Если Docker поставить не удалось, допустим Milvus Lite с пометкой в отчёте.
- Кривая `recall@k` для  $k = 1, 3, 5, 10$  по обоим нарезкам, на одном графике. По ней выберите  $k$  для следующего шага и объясните выбор одной фразой.
- Гибрид: поиск по словам и слияние рангов RRF хотя бы на одной нарезке, таблица «вектор, слова, гибрид».

**Сначала поиск, потом ответы**

Не переходите к генерации, пока не посмотрели на `recall@k`. Если верного куска нет в первых пяти, никакая модель не ответит, а вы потратите деньги на выяснение этого факта. На семинаре одна замена нарезки подняла `recall@1` с 0.5 до 0.9 и не стоила ни цента.

**Шаг 3. RAG-ответ, инструмент агента и замер**

Соберите `rag_answer` с семинара на своём корпусе и инструмент `knowledge_base` для агента из первой домашки: со схемой, описанием и тестом на три случая. Дальше таблица `report` и картинка `money_chart` на всех ваших вопросах. Обязательные строки:

| Конфигурация                                        | Зачем она в таблице                    |
|-----------------------------------------------------|----------------------------------------|
| сильная модель, без инструментов                    | строка, которую надо обогнать          |
| дешёвая модель, живой поиск из первой домашки       | прошлая неделя, для сравнения          |
| дешёвая модель, RAG всегда                          | поиск перед каждым ответом             |
| дешёвая модель, агент с <code>knowledge_base</code> | модель сама решает, когда и что искать |
| средняя модель, RAG всегда                          | сколько добавляет модель посильнее     |

По каждой строке: доля верных, цена вопроса, цена верного ответа. Отдельной таблицей отказы: сколько из десяти вопросов без ответа система честно отклонила и сколько вопросов с ответом отклонила зря. Если у вас свой корпус и живого поиска по нему не было, вторую строку замените на «дешёвая модель, без инструментов».

Разбор провалов: разложите ошибки лучшей конфигурации на две кучки, «поиск не нашёл» и «нашёл, но ответ неверный», и разберите по одному примеру из каждой: что было в контексте и что надо менять, нарезку, запрос, промпт или проверку ответа.

**Шаг 4. Память**

Агент из шага 3 получает две памяти, как на семинаре.

- **Эпизодическая:** журнал реплик в файле, с сессией и временем. В контекст он не идёт.
- **Семантическая:** факты о пользователе, которые дешёвая модель извлекает в конце сессии. Хранятся в отдельной коллекции Milvus, к вопросу достаются несколько ближайших. Новый факт, который отменяет старый, заменяет его, а не ложится рядом.
- **Демонстрация из двух сессий** в ноутбуке: в первой пользователь рассказывает о себе и задаёт вопрос, во второй агент использует то, что узнал, хотя история первой сессии в контекст не попадала. Затем пользователь меняет один из фактов, и после сессии в памяти лежит новый.
- **График токенов:** диалог не меньше чем из десяти реплик двумя способами, «вся история в контексте» и «окно плюс факты». Токены на входе по репликам и суммарная цена обоих вариантов.
- **Тесты без модели:** запись и поиск фактов проверяются без вызова LLM на трёх случаях: пустая память, найденный факт, заменённый факт.

**Что сдавать**

- `README.md`: что за корпус и откуда, как резали и почему; график `recall@k`; таблица гибрида; итоговая таблица и картинка; таблица отказов; два разобранных провала; график токенов; абзац с выводом по тезису курса: стал ли верный ответ дешевле, чем на прошлой неделе, и за счёт чего;
- корпус или скрипт, который его скачивает, вопросы в формате стартового набора;
- ноутбук с выполненными ячейками;
- `docker-compose.yml` для Milvus; кэш эмбедингов можно закоммитить, если он меньше 50 МБ; журнал, данные базы и `.env` в `.gitignore`;

- `.env.example`, `requirements.txt`, ключа в истории коммитов нет.

## Оценка, 10 баллов

| Критерий                                                             | Баллы |
|----------------------------------------------------------------------|-------|
| Репозиторий запускается с чистого клона, ключа в истории нет         | 1     |
| Корпус, вопросы с цитатами, две нарезки, индекс в Milvus с фильтром  | 2     |
| Кривая recall@k по двум нарезкам, гибрид с RRF, обоснованный выбор k | 2     |
| Таблица со всеми обязательными строками, картинка, таблица отказов   | 2     |
| Память: две сессии, замена факта, график токенов, тесты без модели   | 2     |
| Два разобранных провала и честный вывод по тезису                    | 1     |
| Бонус: корпус из PDF, в ответе ссылка на файл и страницу             | +1    |
| Бонус: реранкер или выдача родительского куска, сравнение до и после | +1    |

Как и в прошлый раз, оценка не зависит от того, подтвердился тезис или нет. Зависит от того, честно ли посчитано.

### Бюджет

Эмбединги почти бесплатны: тысяча кусков стоит меньше цента. Деньги уходят на ответы: пять кусков в контексте это 300–400 токенов на входе, и на дешёвой модели вопрос стоит меньше сотой доли цента. Весь прогон семинарского ноутбука обошёлся в 17 центов, из них 12 съела сильная модель. Ваш замер на пятидесяти вопросах в пяти конфигурациях должен уложиться в 30–50 центов. Если счётчик ушёл за два доллара, посмотрите, не считаете ли вы эмбединги заново при каждом запуске и не кладёте ли в контекст страницы целиком.

## Часть 2, опциональная: знания и память вашего проекта

Продолжение опциональной части первой домашки, для тех, кто строит своего агента. Если задача ещё не обсуждена с @artemovma, начните с этого.

### Шаг 1. Что агент должен знать и что помнить

Одна страница `MEMORY.md`. Какие документы и данные нужны агенту, чтобы делать свою работу, где они лежат и как часто меняются. Что агент должен помнить о пользователе между сессиями, а что обязан забывать. Чего в памяти быть не должно никогда: пароли, платёжные данные, чужие персональные данные. Что происходит, когда знания в базе устарели, и кто это замечает.

### Шаг 2. Скилл базы знаний и политика памяти

- Скилл `knowledge_base` поверх документов вашего проекта: схема, описание, метаданные, фильтр, тест на три случая. Требования к скиллам те же, что в первой домашке: тесты запускаются без модели и без ключа.
- Память с правилами из `MEMORY.md`: что записывается, как обновляется, как пользователь может посмотреть и стереть то, что агент о нём знает. Команды «что ты обо мне помнишь» и «забуди это» должны работать.
- Десять вопросов, на которые ваш агент обязан отвечать по базе, и пять, на которые обязан отказаться. Прогон и таблица.

Сдаётся в той же папке `project`, что и скиллы первой недели. Оценивается как бонус, до трёх баллов, и переносится в зачёт финального проекта.

Вопросы по заданию в чат курса. Проблемы с ключом и бюджетом: @artemovma в телеграме. Следующая неделя: фреймворки, критик и планирование, всё на том же агенте и с тем же счётчиком денег.